URI scheme, as described in the previous section. To protect against this attack, the OAuth working group developed the *PKCE standard* (Proof Key for Code Exchange, https://tools.ietf.org/html/rfc7636), pronounced “pixy.” Since then, formal analysis of the OAuth protocol has identified a few theoretical attacks against the authorization code flow. For example, an attacker may be able to obtain a genuine authorization code by interacting with a legitimate client and then using an XSS attack against a victim to replace their authorization code with the attacker’s. Such an attack would be quite hard to pull off but is theoretically possible. It’s therefore recommended that all types of clients use PKCE to strengthen the authorization code flow.

The way PKCE works for a client is quite simple. Before the client redirects the user to the authorization endpoint it generates another random value, known as the PKCE *code verifier*. This value should be generated with high entropy, such as a 32-byte value from a `SecureRandom` object in Java; the PKCE standard requires that the encoded value is at least 43 characters long and a maximum of 128 characters from a restricted set of characters. The client stores the code verifier locally, alongside the state parameter. Rather than sending this value directly to the AS, the client first hashes[7] it using the SHA-256 cryptographic hash function to create a *code challenge (listing* 7.4). The client then adds the code challenge as another query parameter when redirecting to the authorization endpoint.

Listing 7.4 Computing a PKCE code challenge

```
String addPkceChallenge(spark.Request request,
```